

Part II

Robustness

7 Overview: Robustness to What?

This part studies how learning systems behave under perturbations of different kinds. You may have seen a trend of papers with titles like “Quadruply Robust Learning” or “Triply Robust Estimation.” Why are there so many different ways to be robust?¹ To compare and contrast, let’s start with a very general model of what it means to learn or estimate that we can use to introduce some of these ideas at a very high level. The cast of characters is:

- There is a distribution $\mathcal{D} \in \mathfrak{D}$ over some domain $\mathcal{X}$. Here $\mathfrak{D}$ is some class of potential distributions that we consider. In supervised learning sometimes the domain is denoted $\mathcal{X} \times \mathcal{Y}$ where $\mathcal{X}$ is a feature space and $\mathcal{Y}$ is a label space. In ML/AI parlance this is often called the training distribution and in Statistics parlance this is often called the population.
- There is a sample $X = (X_1, \dots, X_n)$ from this distribution. In supervised learning sometimes the samples are denoted (X_i, Y_i) . In ML/AI parlance this is often called the training data.
- There is some quantity or parameter $\theta \in \Theta$ that we are trying to estimate from the sample. In ML/AI parlance this is often called the model parameters and in Statistics parlance this is often called the estimate or estimated parameter.
- There is a loss function $\ell: \Theta \times \mathfrak{D} \rightarrow \mathbb{R}$ that measures how well a parameter θ fits a distribution $\mathcal{D}$. We write this loss as $\ell(\theta; \mathcal{D})$, with the parameter first and the distribution second. In ML/AI parlance this is often called the training loss and in Statistics parlance this is often called the risk or error.
- There is an algorithm $\mathcal{A}$ that takes the sample X and outputs a parameter $\hat{\theta}$. In ML/AI parlance this is often called the learning algorithm or training procedure and in Statistics parlance this is often called the estimator.

The goal is to find a parameter $\hat{\theta}$ that has as little loss as possible with respect to the distribution.

$$\theta^* = \arg \min_{\theta \in \Theta} \ell(\theta; \mathcal{D}).$$

We call the gap between the loss of the estimated parameter $\hat{\theta}$ and the loss of the best parameter θ^* , the excess risk or error. We want to design the algorithm to estimate parameters with low excess risk or error. The simplest way to measure that is

$$\mathbb{E}_{X \sim \mathcal{D}^n} \left[\underbrace{\ell(\mathcal{A}(X); \mathcal{D}) - \ell(\theta^*; \mathcal{D})}_{\text{excess error}} \right],$$

but we can also ask for a guarantee that the excess error is small with high probability rather than just in expectation.

¹I’m actually not entirely sure.

Note that the distribution $\mathcal{D}$ occurs in two places. First, it is what generates the sample that we give to the algorithm (shown in blue). Second, it is the distribution that we use for measuring the loss (shown in red). At the highest possible level

Robustness algorithms have small error when the distribution generating the data and the distribution measuring the loss are different.

Let's try to make this all more concrete. First let's introduce a few common instantiations of the general setup *without robustness*.

7.1 Examples of the Learning Setup

7.1.1 Mean and Covariance Estimation

In the mean estimation problem, the data points are vectors in $\mathbb{R}^d$, and the goal is to estimate the mean $\mu = \mathbb{E}_{X \sim \mathcal{D}}[X]$ of the distribution. The loss function is typically the squared Euclidean error

$$\ell(\hat{\mu}; \mathcal{D}) = \|\hat{\mu} - \mu\|_2^2,$$

where $\hat{\mu} = \mathcal{A}(X)$ is the estimate. In covariance estimation, the goal is instead to estimate the covariance matrix

$$\Sigma = \mathbb{E}_{X \sim \mathcal{D}}[(X - \mu)(X - \mu)^\top].$$

For an estimate $\hat{\Sigma} = \mathcal{A}(X)$, one common loss is the squared Frobenius error

$$\ell(\hat{\Sigma}; \mathcal{D}) = \|\hat{\Sigma} - \Sigma\|_F^2.$$

Without some assumption on $\mathcal{D}$, estimating the mean and covariance from samples is impossible. We therefore restrict $\mathcal{D}$ to a family $\mathfrak{D}$ with additional structure. For example, we could take $\mathfrak{D}$ to be the class of all d -dimensional Gaussian distributions,

$$\mathfrak{D}_{\text{Gauss}} = \{\mathcal{N}(\mu, \Sigma) : \mu \in \mathbb{R}^d, \Sigma \in \mathbb{R}^{d \times d}, \Sigma \succeq 0\}.$$

Here $X \sim \mathcal{N}(\mu, \Sigma)$ means that $X = \mu + AZ$, where $Z \sim \mathcal{N}(0, I)$ has independent standard normal coordinates and $AA^\top = \Sigma$. When $\Sigma \succ 0$, this distribution has density

$$p_{\mu, \Sigma}(x) = \frac{1}{(2\pi)^{d/2} \det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right).$$

7.1.2 Supervised Learning

In supervised learning, the data points are labeled examples $z = (x, y)$ and $\mathcal{D}$ is a distribution over such examples. The loss is typically *decomposable*, or equivalently linear in the distribution:

$$\ell(\theta; \mathcal{D}) = \mathbb{E}_{z \sim \mathcal{D}}[\ell(\theta; z)].$$

The norm is to overload notation and write $\ell(\theta; z)$ for the loss evaluated on the point mass at z , that is, $\ell(\theta; z) = \ell(\theta; \delta_z)$. Thus we evaluate the loss on an individual example as if that example were itself a distribution. Similarly we can treat a set of examples if it were the uniform distribution over that set and evaluate the loss on this set as if it were itself a distribution.

In this setting the norm is also to distinguish between the weights of the model, which we denote θ , and the model itself which we denote f_θ . We more or less interchangeably think of the algorithm as outputting the weights or the model itself depending on which is convenient.

Linear regression. Here $z = (x, y) \in \mathbb{R}^d \times \mathbb{R}$, the parameter $\theta \in \mathbb{R}^d$ defines the predictor $f_\theta(x) = \langle x, \theta \rangle$, and the pointwise squared loss is

$$\ell(\theta; (x, y)) = (y - f_\theta(x))^2.$$

Binary Classification with logistic regression. For binary labels $y \in \{0, 1\}$, the parameter $\theta \in \mathbb{R}^d$ defines the logistic predictor

$$f_\theta(x) = \sigma(\langle x, \theta \rangle), \quad \sigma(t) = \frac{1}{1 + \exp(-t)},$$

where $f_\theta(x)$ is the predicted probability that $y = 1$. The pointwise cross-entropy loss is

$$\ell(\theta; (x, y)) = -y \log f_\theta(x) - (1 - y) \log(1 - f_\theta(x)).$$

Multiclass classification with a neural network. For multiclass classification with k classes, the data points are pairs $(x_i, y_i) \in \mathbb{R}^d \times \{1, \dots, k\}$. A neural network with some specific architecture and weights θ outputs a vector of logits $f_\theta(x_i) \in \mathbb{R}^k$. The cross-entropy loss on an example is the negative log probability assigned to the correct class:

$$\ell(\theta; (x, y)) = -\log \left(\frac{\exp(f_\theta(x)_y)}{\sum_{j=1}^k \exp(f_\theta(x)_j)} \right).$$

7.1.3 Stochastic Optimization

In stochastic optimization, we use the same abusive notation for the loss on a point, a distribution, and a sample. For a sample $X = (X_1, \dots, X_n) \sim \mathcal{D}^n$, define

$$\ell(\theta; \mathcal{D}) = \mathbb{E}_{Z \sim \mathcal{D}} [\ell(\theta; Z)] \quad \text{and} \quad \ell(\theta; X) = \frac{1}{n} \sum_{i=1}^n \ell(\theta; X_i).$$

Thus $\ell(\theta; X)$ treats the sample as its uniform empirical distribution. The goal is to design an algorithm $\mathcal{A}$ whose output $\mathcal{A}(X)$ has small excess risk on the underlying distribution:

$$\mathbb{E}_{X \sim \mathcal{D}^n} [\ell(\mathcal{A}(X); \mathcal{D}) - \ell(\theta^*; \mathcal{D})],$$

where θ^* minimizes $\ell(\theta; \mathcal{D})$ over $\theta \in \Theta$. This general family of stochastic optimization problems captures all of the examples above by choosing the parameter space and loss appropriately.

7.2 Examples of Robustness

As we said, different robustness properties can (very informally, and occasionally imprecisely) be viewed as cases where either the data distribution is perturbed or the test distribution is perturbed or both. These models are typically differentiated by the stories we tell about what is perturbing the distribution and what their objectives are, leading to different formalisms. It's a bug and a feature that the abstract learning setup we use makes all these cases look similar to one another—it can oversimplify and abstract away the real problem we care about, but it also allows us to strip away details to identify common ideas, and transfer ideas from one problem to another.²

²In my opinion, this is a pretty good description of foundational research broadly.

7.2.1 Adversarial Corruption

In the classic model of robust statistics, a clean sample $X \sim \mathcal{D}^n$ is given to an adversary C , which may inspect X and corrupt at most a τ fraction of its points. The algorithm receives only the corrupted sample $\tilde{X} = C(X)$. We call τ the corruption rate. We now measure the expected error when the algorithm is given the corrupted sample, and then evaluated with respect to the worst adversary and the original distribution.

$$\max_{C: \text{corruption rate } \tau} \mathbb{E}_{X \sim \mathcal{D}^n} [\ell(\mathcal{A}(C(X)); \mathcal{D})].$$

A slightly weaker notion of robustness would be to consider adversaries who directly corrupt the distribution $\mathcal{D}$ by creating a mixture

$$\tilde{\mathcal{D}} = (1 - \tau)\mathcal{D} + \tau\mathcal{D}',$$

where $\mathcal{D}'$ is an arbitrary distribution chosen by the adversary. This latter version is the standard model of adversarial corruption in robust statistics [Hub64].

Note that this definition considers an omniscient adversary who knows the distribution, has full knowledge of the dataset, can both insert and delete points in the dataset, and has unbounded computational power. Depending on who we think the attacker represents we might relax these choices in different ways, but it's sort of remarkable that you can protect against such a powerful adversary at all.

7.2.2 Targeted Data Poisoning

In the adversarial corruption model, the adversary and the algorithm have exactly opposite goals: the adversary tries to maximize the population loss of the model trained on the corrupted sample. In the data poisoning literature, these are sometimes called availability attacks. However, many real attacks have more specific objectives, such as causing the algorithm to misclassify a particular input. These are typically called targeted attacks.

For a concrete example in our framework, fix a target distribution $\mathcal{D}_{\text{target}}$. The adversary chooses C to maximize the loss of the corrupted-data model on this target distribution:

$$\mathbb{E}_{X \sim \mathcal{D}^n} [\ell(\mathcal{A}(C(X)); \mathcal{D}_{\text{target}})].$$

Unlike an availability attack, this objective only cares about the model's behavior on $\mathcal{D}_{\text{target}}$, rather than its overall loss on the original population $\mathcal{D}$.

From the algorithm's perspective, a robust algorithm should make it impossible for the adversary to substantially increase the loss on this target distribution, leading to the objective

$$\max_{C: \text{corruption rate } \tau} \mathbb{E}_{X \sim \mathcal{D}^n} [\ell(\mathcal{A}(C(X)); \mathcal{D}_{\text{target}}) - \ell(\mathcal{A}(X); \mathcal{D}_{\text{target}})].$$

This stability objective is typically paired with the requirement that the clean loss on the original distribution $\ell(\mathcal{A}(X); \mathcal{D})$ is itself small.

7.2.3 Distribution Shift

In distribution shift, we train the algorithm on a sample $X \sim \mathcal{D}^n$ from a training distribution $\mathcal{D}$, but test the resulting model on a different distribution $\tilde{\mathcal{D}}$. In our framework, its expected test loss is

$$\mathbb{E}_{X \sim \mathcal{D}^n} [\ell(\mathcal{A}(X); \tilde{\mathcal{D}})].$$

Note that in some sense the distribution shift setting and the adversarial corruption setting are similar, although with the role of $\mathcal{D}$ and $\tilde{\mathcal{D}}$ swapped. Distribution shift looks harder in that we actually allow the attacker to change the entire distribution rather than just a fraction of the sample. But in order to be robust in this way, we need to make some other kind of assumptions about how $\mathcal{D}$ and $\tilde{\mathcal{D}}$ are similar.

7.2.4 Adversarial Examples

Adversarial examples move the corruption from training time to test time. The training sample X is unchanged, but for each test example $(x, y) \sim \mathcal{D}$, the adversary may replace the feature x by a nearby feature $\tilde{x}$ while the true label y remains fixed. The goal is to design an algorithm $\mathcal{A}$ with small expected worst-case test loss

$$\mathbb{E}_{X \sim \mathcal{D}^n} \mathbb{E}_{(x, y) \sim \mathcal{D}} \left[\max_{\tilde{x}: d(\tilde{x}, x) \leq \rho} \ell(\mathcal{A}(X); (\tilde{x}, y)) \right].$$

We can also pair this concept with targeted data poisoning where we allow an attacker to both corrupt the training data and also the target data.

8 Robust Mean Estimation

8.1 Means and Medians

We begin with the simplest estimation problem: estimating the mean $\mu = \mathbb{E}_{X \sim \mathcal{D}}[X]$ of a distribution $\mathcal{D}$ over $\mathbb{R}$. We measure the quality of an estimate $\hat{\mu}$ using squared-error loss

$$\ell(\hat{\mu}; \mathcal{D}) = (\hat{\mu} - \mu)^2.$$

Given a clean sample $X = (X_1, \dots, X_n) \sim \mathcal{D}^n$, the most natural estimator is the sample mean

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i.$$

The sample mean is not robust to adversarial corruption. Inserting one point can change the sample mean by an arbitrarily large amount.

The sample median is the classical robust alternative to the mean.

$$\text{med}(X) = \begin{cases} X_{(n/2)} & \text{if } n \text{ is even,} \\ X_{((n+1)/2)} & \text{if } n \text{ is odd,} \end{cases}$$

where $X_{(1)} \leq \dots \leq X_{(n)}$ are the order statistics of the sample.

It is much less sensitive to extreme values, but without assumptions on the clean distribution it is not uniformly robust either. For example, the median of the sample $(0, 0, B)$ is 0, but after replacing 0 by B , the median is now B .

In fact, without any restriction on $\mathcal{D}$, robust mean estimation is impossible for every estimator. To see the basic obstruction consider

$$\mathcal{D}_0 = \delta_0 \quad \text{and} \quad \mathcal{D}_1 = (1 - \tau)\delta_0 + \tau\delta_B.$$

These distributions have means 0 and τB , which can be arbitrarily far apart. Nevertheless, an adversary in the mixture corruption model can make them identical by corrupting $\mathcal{D}_1$ to $\mathcal{D}_0$.

Intuitively, robust estimators try to identify and remove the corrupted data, at least in cases where that data is harmful. But there is no way to identify the corrupted points without some assumptions about what the uncorrupted points should look like. We will start with the natural starting point that the distribution is Gaussian: $\mathcal{D} = \mathcal{N}(\mu, \sigma^2)$. In this setting the sample median is robust because a Gaussian places a substantial amount of probability mass near its median, which is also its mean. An adversary must therefore corrupt a substantial fraction of the sample to move the median a substantial distance.

Theorem 8.1. *Let $X_1, \dots, X_n \sim \mathcal{N}(\mu, \sigma^2)$ be i.i.d. samples, and let $\tilde{X}$ be a τ -corruption of X . Then with probability at least $1 - \delta$ over the choice of the clean sample X ,*

$$(\mu_{\text{med}}(\tilde{X}) - \mu)^2 = O\left(\frac{\sigma^2 \log(1/\delta)}{n} + \sigma^2 \tau^2\right)$$

Proof sketch. Intuitively, the sorted clean sample lies near the $1/n, 2/n, \dots, n/n$ quantiles of the distribution. Corrupting at most τn points can move the median by at most τn positions in this sorted list, so the median of the corrupted sample lies between roughly the $(1/2 - \tau)$ and $(1/2 + \tau)$ quantiles of the clean distribution. Finally, if Φ is the standard Gaussian cumulative distribution function, then $\Phi^{-1}(1/2 + u) = O(u)$ for u bounded away from $1/2$. Thus the Gaussian quantiles at probability $1/2 \pm u$ are $\mu \pm O(\sigma u)$, and the total quantile error translates into

$$|\mu_{\text{med}}(\tilde{X}) - \mu| = O\left(\sigma \left(\tau + \sqrt{\frac{\log(1/\delta)}{n}}\right)\right).$$

Squaring the bound gives the result. The other term that is not $\sigma^2 \tau^2$ accounts for the noise due to random sampling. $\square$

The trimmed mean. The trimmed mean more explicitly implements the intuition of filtering outliers. Fix a constant $\gamma \in (0, 1/2)$, sort the corrupted sample as $\tilde{X}_{(1)} \leq \dots \leq \tilde{X}_{(n)}$, discard the largest and smallest $\lfloor \gamma n \rfloor$ points, and average the rest:

$$\mu_{\text{trim}}(\tilde{X}) = \frac{1}{n - 2\lfloor \gamma n \rfloor} \sum_{i=\lfloor \gamma n \rfloor + 1}^{n - \lfloor \gamma n \rfloor} \tilde{X}_{(i)}.$$

For Gaussian data and a corruption rate τ sufficiently smaller than γ , the remaining points lie within $O(\sigma)$ of μ , so the corrupted points that survive trimming can shift the estimate by only $O(\sigma \tau)$. Accounting for sampling error gives the same qualitative guarantee as the median:

$$(\mu_{\text{trim}}(\tilde{X}) - \mu)^2 = O\left(\frac{\sigma^2 \log(1/\delta)}{n} + \sigma^2 \tau^2\right)$$

with probability at least $1 - \delta$.